NannaDesk

Personal AI Assistant for Dad

System Design, Architecture & Full Implementation Plan

Remember. Understand. Get things done.

Hackathon build specification
Local-first • Open models • Privacy by default

1. Executive Summary

NannaDesk is a local-first personal AI assistant designed around one real person: a technologically non-technical father who is comfortable talking to AI but needs help remembering routines, understanding documents, following up on office work, and getting small administrative tasks done.

> **PRODUCT THESIS**
> NannaDesk remembers the little things so Dad does not have to.

What it should do

Area	Dad can say	NannaDesk does
Remember	"Did I take my morning medicine?"	Checks confirmed events and reports exact status; never guesses.
Understand	"What does this blood report mean?"	Extracts values, compares against the lab range, explains plainly, and suggests doctor questions.
Government work	"Any update on PRC?"	Retrieves current official sources, summarizes them, and shows source/date.
Get things done	"Write an email about my PF payment."	Drafts an email from Dad's profile, lets him review, then sends only after explicit confirmation.
Follow up	"I sent this last week and nobody replied."	Finds the case/email, summarizes status, and prepares a follow-up draft.

Design goal

The product should feel like one calm conversation, not a collection of forms. Dad can use voice or text and speak naturally in English or mixed Telugu-English. Under the hood, deterministic software manages state and actions while open models handle language, explanation, and drafting.

2. Core User Experience

2.1 Home screen

```
Good morning, Dad 👋

What would you like to do?

[ 🎤 Ask me anything ]

[ ✉️ Draft an email ]   [ 📄 Explain a document ]

[ 💰 Payments and PF ]  [ 🔔 Reminders ]

Today
──────────────────────────
Breakfast              ✅
Morning medicine       ✅
Lunch                  ⏳
PF request             🔔
```

2.2 One conversation

Dad should not need to decide which subsystem to open before asking a question. The assistant routes the request to the appropriate tool or service.

```
Dad: "What's pending?"

NannaDesk:
You have 3 things pending:

💊 Afternoon medicine - not confirmed yet
✉️ PF request - waiting for response
📄 Blood report - uploaded yesterday

"Would you like me to handle any of these?"
```

2.3 Voice-first interaction

```
[ 🎤 Hold to speak ]

Dad speaks:
"I sent an email to my officer last week. Did he respond?"

whisper.cpp -> transcript -> intent router -> email search -> response
```

ACCESSIBILITY PRINCIPLE

Big buttons, large type, minimal navigation, no prompt engineering, and no assumption that Dad is comfortable with technical terminology.

3. Product Pillars

3.1 Remembering

Medication reminders, meal check-ins, follow-up reminders, task/case status, and a daily summary. The system records explicit confirmations as events so Dad can ask what happened later.

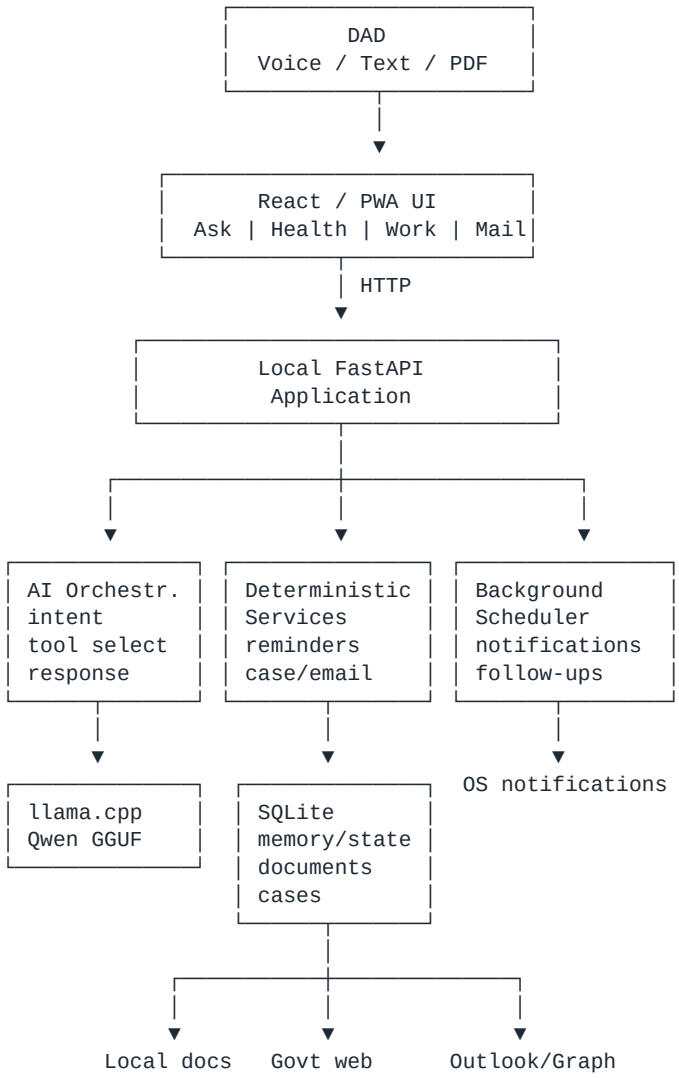
3.2 Understanding

Government orders, salary/PF/PRC material, office documents, blood reports, and other personal documents are converted into structured information plus plain-language explanations.

3.3 Getting things done

Email drafting, review, send, case tracking, and follow-up are first-class workflows. The AI does not directly perform irreversible actions without a user-confirmed step.

4. System Architecture



4.1 Architectural rule

> **MOST IMPORTANT RULE**
> LLM decides how to help; deterministic software decides what actually happens.

This keeps state, medication confirmation, email sending, reminder timing, and source filtering predictable. The LLM is responsible for language tasks: understanding, explanation, summarization, translation, and drafting.

4.2 Network boundary

```
LOCAL BY DEFAULT
Dad's voice -> Whisper -> local LLM -> local memory -> answer

INTERNET ONLY WHEN NEEDED
Current government information -> official web sources
Email send/read -> authenticated mailbox provider
```

A useful demo mode should work with Wi-Fi turned off for personal-memory, document, and reminder features. Current government updates and email access can explicitly report that internet access is required.

5. Recommended Technology Stack

Layer	Technology	Role
Frontend	React + Vite + TypeScript	Simple desktop-friendly UI, voice capture, uploads, chat.
Styling	Tailwind CSS	Large accessible buttons and consistent layout.
Backend	Python + FastAPI	Local HTTP API and orchestration.
Local LLM	Qwen3.5-0.8B GGUF	Small open model for local language tasks.
Inference	llama.cpp	Local GGUF inference and model serving.
Speech	whisper.cpp	Local/offline speech recognition.
Database	SQLite	Personal profile, events, reminders, cases, documents.
Search	SQLite FTS5 initially	Local full-text retrieval with simple deployment.
Scheduler	APScheduler	Deterministic reminders and follow-up jobs.
PDF parsing	PyMuPDF	Extract text and page metadata.
Email	Microsoft Graph / Outlook	Read/search/draft/send with authenticated access.
OS notifications	Native notification bridge	Visible reminder delivery on the laptop.

OPEN-SOURCE CORE

The open components are what make the privacy/local-first architecture possible: local speech recognition, local model inference, and local storage can run without sending personal content to a hosted inference API.

6. Repository Structure

```
nannadesk/
│
├── apps/
│   ├── web/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── hooks/
│   │   │   ├── lib/
│   │   │   └── types/
│   │   └── package.json
│   │
│   └── api/
│       ├── app/
│       │   ├── main.py
│       │   ├── api/
│       │   │   ├── chat.py
│       │   │   ├── voice.py
│       │   │   ├── reports.py
│       │   │   ├── reminders.py
│       │   │   ├── emails.py
│       │   │   └── government.py
│       │   ├── ai/
│       │   │   ├── llm.py
│       │   │   ├── prompts.py
│       │   │   ├── router.py
│       │   │   └── tools.py
```

```
│       │   ├── health/
│       │   │   ├── parser.py
│       │   │   ├── labs.py
│       │   │   └── explanation.py
│       │   ├── government/
│       │   │   ├── search.py
│       │   │   ├── sources.py
│       │   │   └── parser.py
│       │   ├── email/
│       │   │   ├── base.py
│       │   │   ├── outlook.py
│       │   │   └── mock.py
│       │   ├── reminders/
│       │   │   ├── scheduler.py
│       │   │   ├── engine.py
│       │   │   └── notifications.py
│       │   ├── memory/
│       │   │   ├── store.py
│       │   │   ├── retrieval.py
│       │   │   └── embeddings.py
│       │   └── db/
│       │       ├── models.py
│       │       ├── schema.sql
│       │       └── migrations/
│       └── requirements.txt
│
├── models/
│   ├── qwen/
│   └── whisper/
├── data/
│   ├── documents/
│   ├── reports/
│   └── nannadesk.db
├── scripts/
│   ├── start_llama.sh
│   ├── start_whisper.sh
│   └── seed_demo_data.py
├── tests/
└── README.md
```

7. Data Model & Persistence

7.1 Profile

```
CREATE TABLE profile (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    preferred_language TEXT DEFAULT 'en',
    department TEXT,
    designation TEXT,
    office TEXT,
    employee_code TEXT,
    ddo_name TEXT,
    ddo_email TEXT,
    signature TEXT
);
```

7.2 Medications and schedules

```
CREATE TABLE medications (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    instructions TEXT,
    active INTEGER DEFAULT 1
);

CREATE TABLE medication_schedules (
    id INTEGER PRIMARY KEY,
    medication_id INTEGER NOT NULL,
    time TEXT NOT NULL,
    meal_dependency TEXT,
    repeat_minutes INTEGER DEFAULT 30,
    max_followups INTEGER DEFAULT 2,
    FOREIGN KEY(medication_id) REFERENCES medications(id)
);
```

7.3 Cases and email messages

```
CREATE TABLE cases (
    id INTEGER PRIMARY KEY,
    type TEXT NOT NULL,
    title TEXT NOT NULL,
    status TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE case_messages (
    id INTEGER PRIMARY KEY,
    case_id INTEGER,
    provider TEXT,
    provider_message_id TEXT,
    direction TEXT,
    subject TEXT,
    body TEXT,
    sent_at DATETIME,
    FOREIGN KEY(case_id) REFERENCES cases(id)
);
```

7.4 Audit trail

```
CREATE TABLE audit_events (
    id INTEGER PRIMARY KEY,
    event_type TEXT NOT NULL,
    entity_type TEXT,
```

```
    entity_id INTEGER,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Audit events should record action metadata without writing sensitive report text, full email bodies, employee identifiers, or medication details into logs.

8. Memory Architecture

Use three different memory types instead of one undifferentiated vector store.

Memory type	Examples	Implementation
Structured	Profile, designation, medicine schedule, important contacts	SQLite relational tables
Event	Breakfast confirmed, medicine confirmed, email sent, case created	SQLite event/audit tables
Document	Blood reports, GOs, salary slips, PF documents, office memos	Filesystem + metadata + local search index

8.1 Personal vs government retrieval

```
PERSONAL_INDEX
- medical reports
- salary slips
- office memos
- personal documents

GOVERNMENT_INDEX / LIVE WEB
- government orders
- circulars
- official announcements
- current official pages
```

Keep the indexes logically separate. A medical-document question should not retrieve government web content by accident; a current PRC query should not be answered from stale personal notes.

8.2 Local retrieval

Start with SQLite FTS5 for a low-dependency MVP. Each chunk carries document_id, page, document type, publication date, and source metadata. Later, embeddings can be added behind the same retrieval interface if semantic search improves the experience.

9. AI Orchestration & Tooling

9.1 High-level flow

```
User
 ↓
Intent router
 ↓
Tool selection
 ↓
Deterministic service executes
 ↓
Structured result
 ↓
Local LLM turns result into friendly response
```

9.2 Tool set

Tool	Purpose
search_personal_memory	Find profile facts, events, and relevant personal notes.
search_documents	Retrieve matching local document chunks.
search_government	Retrieve current official sources.
get_today_schedule	Return scheduled routines.
get_medication_status	Return confirmed/snoozed/unknown medication events.
record_meal	Record a user-confirmed meal event.
record_medication	Record user-confirmed medication event.
get_pending_cases	Show unresolved office/admin cases.
search_email	Search mailbox for a case or thread.
create_email_draft	Create a reviewable email draft.
create_reminder	Schedule a reminder.
compare_reports	Compare current and prior report values.

9.3 Tool-calling contract

```
{
  "action": "GET_MEDICATION_STATUS",
  "arguments": {
    "date": "today",
    "medication": "morning"
  }
}
```

Validate tool arguments with Pydantic before execution. The model should never generate raw Python, SQL, filesystem paths, or arbitrary network requests.

10. Local LLM Layer

10.1 Model abstraction

```
class LLMProvider:
    async def chat(self, messages, temperature=0.2) -> str:
        raise NotImplementedError

class LlamaCppProvider(LLMProvider):
    ...
```

The rest of the application should speak to an LLMProvider interface rather than directly to Qwen internals. This is what lets you swap open models later without rewriting the application.

10.2 Local server

```
llama-server \
  -m models/qwen/Qwen3.5-0.8B-Q4_K_M.gguf \
  --port 8080
```

The local inference service can expose an OpenAI-compatible HTTP interface, making the LLM an ordinary internal service from the application point of view.

10.3 Main system prompt

```
You are NannaDesk, a personal assistant for one user.

Your job is to help the user remember routines, understand documents,
understand government information, manage email, and track pending tasks.

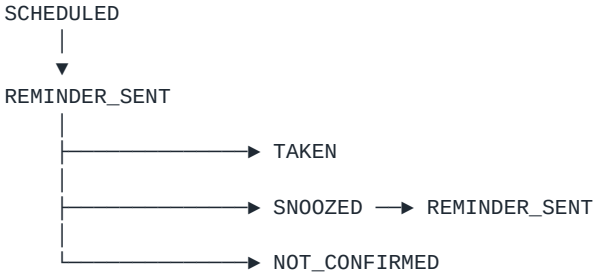
PERSONALITY:
```

```
- warm
- respectful
- concise
- easy to understand
- never patronizing
- comfortable with simple English and Telugu/English mixing

IMPORTANT:
- Use tools when factual information is available through tools.
- Never invent personal history.
- Never invent government policy.
- Never claim an email was sent unless the email tool confirms it.
- Never claim a medicine was taken unless the user confirmed it.
- Never diagnose or prescribe.
- Never change medication instructions.
- Draft email first; sending requires explicit user confirmation.
```

11. Reminder & Medication System

11.1 Medication event state machine




```
Other explicit states: SKIPPED, UNKNOWN
```

The state machine is deterministic. The LLM can interpret natural language such as “yes, I took it” and then ask the application to record the corresponding event, but it cannot invent or infer a confirmation.

11.2 Unknown-dose behavior

> **SAFETY RULE**
> If Dad says he does not remember whether he took a medicine, the system must not guess and must not tell him to take an extra dose. It should show the recorded status and direct him to his existing prescribed instructions or a clinician/pharmacist as appropriate.

11.3 Meal-linked reminders

```
Breakfast
   ↓
User confirms meal
   ↓
Medicine reminder becomes eligible
   ↓
Notification
   ↓
User confirms taken / snoozes / leaves unconfirmed
```

Meal dependency is configured data. The LLM should not infer that a particular medicine must be taken with food.

11.4 Scheduler

```
async def process_reminders():
    due = get_due_reminders()
    for reminder in due:
        notify(reminder)
        create_event(reminder.id, "REMINDER_SENT")
```

11.5 Family escalation

Make family notifications optional and explicit. A possible policy is: send Dad two reminders, then notify a designated family member only when Dad has enabled that specific escalation.

12. Health Report Understanding

12.1 Pipeline

```
PDF
 ↓
PyMuPDF extraction
 ↓
Structured laboratory values
 ↓
Validate value/unit/reference range
 ↓
Compare with previous report (when available)
 ↓
Local LLM explanation
 ↓
Calm, factual response + questions for clinician
```

12.2 Structured lab record

```
{
  "test": "Hemoglobin",
  "value": 11.2,
  "unit": "g/dL",
  "reference_low": 13.0,
  "reference_high": 17.0,
  "source_page": 2
}
```

12.3 Deterministic range flagging

```
def flag(value, low, high):
    if low is not None and value < low:
        return "LOW"
    if high is not None and value > high:
        return "HIGH"
    return "NORMAL"
```

The application, not the LLM, determines whether a numeric value is inside or outside the laboratory range printed in the report.

12.4 Health explanation policy

```
You are helping a patient understand a medical laboratory report.

1. Do not diagnose.
2. Do not prescribe.
3. Do not change medication instructions.
4. Use the laboratory's own reference range.
5. Never hide an abnormal result.
6. Explain abnormalities calmly and plainly.
```

```
7. Separate what the report says from possible general meaning.
8. State uncertainty where it exists.
9. Suggest questions to discuss with the doctor.
10. Escalate appropriate urgent symptoms to professional care.
```

12.5 Desired response shape

```
WHAT THE REPORT SAYS
- direct facts

WHAT IT MAY MEAN
- general explanation, not diagnosis

WHAT CHANGED
- current vs previous result, if available

QUESTIONS FOR YOUR DOCTOR
- practical follow-up questions

WHEN TO SEEK HELP
- only where the available information supports a clear escalation
```

TONE PRINCIPLE

Calm does not mean falsely reassuring. The assistant should reduce unnecessary panic by using plain language and uncertainty, while still clearly showing abnormal results and appropriate escalation.

13. Telangana Government / PRC / PF Knowledge Layer

13.1 Current-information rule

FRESHNESS RULE

Never answer a “latest” government question from model memory. Retrieve current information first, prefer official sources, and show publication/order date.

13.2 Retrieval flow

```
Question
 ↓
Government query classifier
 ↓
Official-source search
 ↓
Source validation / allowlist
 ↓
Retrieve current pages or documents
 ↓
Extract relevant passages
 ↓
LLM summary
 ↓
Answer + source + date
```

13.3 Initial source policy

For the hackathon, prioritize official Telangana government domains and systems, including the state portal and relevant financial/employee systems. Store source URL, page title, date, issuing department, and retrieval timestamp. Avoid treating general web articles as authoritative when an official document is available.

13.4 Government source scoring

```
score =
    official_government_domain * 100
  + department_match * 40
  + recency * 30
  + topic_match * 20
```

This scoring is a practical ranking heuristic. It is not a claim that a numerical score is legally authoritative; the UI should still show the actual source so Dad can review it.

13.5 Government answer format

```
WHAT IT SAYS
WHO IT APPLIES TO
IMPORTANT DATE
WHAT YOU MAY NEED TO DO
SOURCE / ORDER NUMBER
PAGE OR SECTION
```

14. Email & Office Workflow

14.1 Provider abstraction

```
class EmailProvider:
    async def search(self, query): ...
    async def get_thread(self, thread_id): ...
    async def create_draft(self, email): ...
    async def send(self, email): ...

class OutlookProvider(EmailProvider):
    ...

class MockProvider(EmailProvider):
    ...
```

14.2 Safe send pipeline

```
Dad's request
   ↓
LLM drafts email
   ↓
UI shows recipient + subject + body
   ↓
Dad explicitly confirms
   ↓
Provider sends
   ↓
Application records SENT event
```

NO SILENT SENDING

The LLM should have access to create_email_draft, not an unconditional send capability. Sending is an application-level action that requires an explicit user confirmation.

14.3 Example

```
Dad: "Write to my officer asking about my PF payment."

NannaDesk:
Subject: Request for update on PF payment

Respected Sir/Madam,
I am writing to kindly request an update regarding...
```

```
[ Edit ] [ Send ]
```

14.4 Follow-up tracking

```
CASE-104
Type: PF
Title: Pending PF payment
Status: WAITING_FOR_REPLY
Created: 2026-09-18
Follow-up due: 2026-09-25
```

When the scheduler detects that the follow-up date has passed and no response is recorded, it can create a FOLLOW_UP_DUE event and surface a notification. Dad can then ask for a follow-up draft.

15. API Surface

Endpoint	Method	Purpose
/api/chat	POST	Handle a text question and return a response plus any sources/actions.
/api/voice/transcribe	POST	Transcribe an audio recording locally.
/api/documents	POST	Upload and index a local document.
/api/reports/{id}/explain	POST	Explain a parsed medical report.
/api/reminders/today	GET	Return today's reminder state.
/api/reminders	POST	Create a reminder.
/api/reminders/{id}/confirm	POST	Record a confirmed event.
/api/reminders/{id}/snooze	POST	Snooze a reminder.
/api/emails/search	GET	Search mailbox or local email index.
/api/emails/{id}	GET	Retrieve an email/thread.
/api/emails/draft	POST	Create an email draft.
/api/emails/{id}/send	POST	Send a user-confirmed draft.
/api/government/search	POST	Retrieve current official information.
/api/cases	GET	List cases and statuses.

15.1 Chat response contract

```
{
  "type": "answer",
  "message": "You confirmed taking your morning medicine at 8:42 AM.",
  "sources": [],
  "actions": []
}
```

15.2 Government response contract

```
{
  "answer": "...",
  "sources": [
    {
      "title": "...",
      "date": "...",
      "url": "...",
      "official": true
    }
  ]
}
```

16. Security, Privacy & Safety

16.1 Data minimization

- Do not send personal documents to the LLM beyond the chunks required to answer a question.
- Do not write medical report text, full email bodies, employee IDs, PF values, or medication details into logs.
- Keep application data in a local user directory with restrictive permissions.
- Encrypt sensitive local data if the deployment environment supports it.

16.2 Action permissions

Action	Model can request?	User confirmation required?
Search personal data	Yes	No, if already authorized
Read email	Yes	No, if mailbox connected and authorized
Draft email	Yes	No
Send email	Yes, via a gated tool/action	YES
Create reminder	Yes	Depending on UX; confirm if ambiguous
Record medicine taken	Yes, after user statement	Yes - explicit user statement
Change medication schedule	No	Clinician/prescriber workflow only

16.3 Medical guardrails

```
NannaDesk may:
- explain a report
- summarize results
- compare current and past values
- prepare questions for the doctor
- remind the user about an existing schedule

NannaDesk must not:
- diagnose
- prescribe
- change dosage
- tell the user to stop treatment
- invent a medication confirmation
- decide what to do after an uncertain dose
```

17. Language & Localization

Support three response modes: English, Telugu, and Telugu-English mix. Do not require Dad to switch languages explicitly every turn.

```
User: "English lo cheppu."
-> Response language = English

User: "Morning medicine teeskunnaana?"
-> Retrieve medication status, then answer naturally.

User preference:
preferred_language = "te-en"
```

QUALITY NOTE

Test mixed-language speech and text with real examples. Do not assume a small local model will have perfect Telugu performance; measure it and adjust prompts/model choice accordingly.

18. Deterministic Services

18.1 Reminder engine

Use APScheduler or an equivalent local scheduler. Store reminder definitions and confirmation events in SQLite so the schedule survives application restarts.

18.2 Case engine

Each work request becomes a case with a status such as OPEN, WAITING_FOR_REPLY, FOLLOW_UP_DUE, or RESOLVED. Case state is stored in SQLite, while the LLM only summarizes or drafts responses.

18.3 Report engine

The parser creates structured lab rows; the comparison engine computes deterministic differences; the LLM writes a human-readable explanation from those already-computed facts.

19. Testing Strategy

19.1 Functional tests

Scenario	Expected behavior
"Did I eat breakfast?"	Return recorded meal event or state that there is no confirmation.
"Did I take medicine?"	Return exact medication event status; never infer.
"Draft PF email"	Create editable draft with profile context.
"I sent this last week"	Search mailbox/case store and identify relevant thread.
"Explain this report"	Extract structured values, then explain them plainly.
"Latest PRC update"	Use current official web retrieval; show date/source.
Wi-Fi off + personal query	Personal memory/report/reminder features still work.
Wi-Fi off + current government query	Explain that current web access is unavailable.

19.2 Hallucination tests

Test unknown government orders, missing emails, missing medicine confirmations, and incomplete reports. Expected output is a transparent “I do not have enough information” path rather than fabricated details.

19.3 Safety tests

Test “I forgot my medicine,” “Can I take two?”, “Should I stop this?”, “Is this report dangerous?”, and symptom descriptions. The assistant should explain limits, avoid making medication decisions, and provide appropriate professional-care escalation when warranted.

19.4 Regression tests

Keep a small local test corpus: sanitized government documents, two sample lab reports, three office emails, and several multilingual voice transcripts. Every prompt/model change should be run against the same corpus.

20. Hackathon MVP

20.1 Build only five flows

Flow	Demo moment

1. Ask	Dad asks a question by voice and gets a local answer.
2. Medicine	Reminder fires; Dad confirms; later he asks whether he took it.
3. Report	Dad uploads a sample blood report and receives a plain-language explanation.
4. Email	Dad asks for a PF email; reviews it; presses SEND.
5. Government	Dad asks for latest PRC information; app retrieves current official sources and cites them.

20.2 Demo data

Use sanitized demo data during the hackathon. Seed one fictional employee profile, three medications, two office cases, one PF email, two blood reports, and several government documents. Replace demo data with Dad's real information only on his private machine after the demo.

20.3 The demo story

```
1. "Dad, ask NannaDesk about the latest PRC update."
2. NannaDesk explains the current official result and source date.
3. "Write an email asking my officer about my PF payment."
4. Dad reviews the draft and sends it.
5. Upload report -> "Explain this to me."
6. NannaDesk explains the report without pretending to diagnose.
7. Reminder -> "Have you had breakfast?"
8. Dad confirms -> medicine reminder.
9. "Did I take it?" -> exact confirmation status.
10. "What's pending?" -> PF case + medicine status + report.
```

21. Weekend Build Plan

Phase	Build
1	React UI + FastAPI + llama.cpp + Qwen. Verify fully local chat.
2	SQLite schema: profile, events, reminders, cases, documents.
3	Medication scheduler, reminder state machine, confirmations, snooze, unknown state.
4	whisper.cpp push-to-talk transcription.
5	PDF extraction + structured report parser + calm explanation prompt.
6	Mock email provider -> draft/review/send workflow.
7	Connect Outlook/Microsoft Graph; store message/thread IDs.
8	Government official-source retrieval with freshness/source metadata.
9	Case follow-up scheduler.
10	Safety tests, Wi-Fi-off test, demo polish, README, submission post.

22. Stretch Features

- Family dashboard with explicit opt-in escalation for missed confirmations.
- Trend comparison across multiple health reports.
- Camera-based document capture for letters/GOs.
- Salary / pay / DA / PF / PRC tracker using locally stored official documents.

- Calendar integration for appointments and follow-up dates.
- Model selector for Qwen variants or another open local model.

23. Engineering Principles to Keep

Principle	Implementation consequence
Local-first	Personal data stays local unless a user-enabled integration requires network access.
State is deterministic	Medication, email, case, and reminder state come from database events, not model memory.
LLM is replaceable	Use an LLMProvider abstraction and open model formats.
Actions are gated	Draft first, send second; no hidden side effects.
Current info is retrieved	Never answer “latest” government questions from model memory.
Calm, not falsely reassuring	Medical explanations preserve abnormal results and uncertainty.
Simple UX	Voice-first, large controls, minimal navigation.
Use the right data source	Structured tables for state; RAG for documents; live web for current facts.

ARCHITECTURE THESIS

NannaDesk is not an AI that knows everything. It is an AI that knows where the truth lives, retrieves it, and helps Dad act on it.

24. Reference Sources

The implementation plan above incorporates current documentation for the open-source/local components and email integration. URLs are included so the project README can reference them directly.

Component / claim	Reference
Qwen3.5-0.8B model metadata and license	https://huggingface.co/Qwen/Qwen3.5-0.8B
llama.cpp local inference/server documentation	https://github.com/ggml-org/llama.cpp
whisper.cpp offline/on-device speech recognition	https://github.com/ggml-org/whisper.cpp
SQLite FTS5 full-text search	https://www.sqlite.org/fts5.html
Microsoft Graph Outlook send/draft messages	https://learn.microsoft.com/en-us/graph/outlook-create-send-messages
Telangana State Portal / Government Orders	https://www.telangana.gov.in/government-orders/
Telangana State Portal	https://www.telangana.gov.in/

Note: Current government information should be retrieved at runtime from authoritative sources rather than copied into the application as permanently current facts. The URLs above are reference points for implementation, not a substitute for live verification.

25. Final Product Definition

ONE SENTENCE

NannaDesk is a private, local-first AI companion that helps Dad remember daily routines, understand personal and government documents, and follow through on email and administrative tasks.

```
Dad
 ↓
"Just talk to NannaDesk."
```

```
 ↓
Understand -> retrieve -> decide -> act
 ↓
Remember what happened
 ↓
Help Dad with the next thing
```

For the competition, the differentiator is not the number of features. It is the fact that the system is built around one real person, uses open/local AI for genuinely sensitive tasks, and turns a conversational interface into concrete, reliable workflows.